

Lab 1.1: Graph Regularization

In this lab, you will learn how to perform graph-based machine learning via regularization techniques. The specific objectives are:

- Implement different graph-based regularization techniques to perform node-wise signal regression.
- Analyze the trade-off between fitting and regularization.
- Explain the performance of particular regularizer.

Completion requirements

- Showing the graph sparsity as a function of abstract network hyper-parameters.
- Implementing and analyzing two regularizers for signal denoising and reconstruction.

Dataset and problem definition

To achieve the above objectives, we will work with a real-world weather dataset called **Molene**. This dataset contains weather temperature over 32 distinct stations in France recorded for a month with hourly resolution. We will treat each time instant as a graph signal and aim to recover it from noisy measurements and signals with missing values. For this, we will compare two regularization techniques:

- Tikhonov regularizer

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{y} - \mathbf{x}\|_2^2 + \alpha \mathbf{x}^T \mathbf{L} \mathbf{x}$$

- Trend filtering.

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{y} - \mathbf{x}\|_2^2 + \beta \|\mathbf{B}^T \mathbf{x}\|_1,$$

where $\mathbf{B}$ is the incidence matrix of the graph.

Libraries

- Numpy
- Pandas
- PyGSP or NetwrokX (or any other viable libraries)
- Scikit-learn
- Matplotlib
- CVXPY and CVXOPT (or any other optimization solver library)

```
In [ ]: import numpy as np
import pandas as pd
from pygsp import graphs, plotting
import matplotlib.pyplot as plt
import cvxpy as cp
import cvxopt as cvxopt
```

Building the graph

We can treat each sensor as a node of a graph and the temperatures as signals over these nodes. These sensors have also metadata such as their geolocation. We can therefore build an abstract network connecting them. This network can for instance be build using these metadata or using a portion of the time series and connecting nodes based on their statistical dependencies in them (e.g. Pearson correlation). In either case, you could use different criteria to connect the nodes such as k -nearest neighbors, ϵ -ball graph, etc. *It is up to you to build any graph from any data*

Task 1

Build a **connected** and **sparse** graph between sensors. Choose your hyperparameters to achieve at least 80% sparsity ($\frac{\text{zeros of } \mathbf{A}}{N^2}$). If you cannot ensure a connected graph with such a sparsity factor it can also be less sparse. Tune your hyper parameters accordingly.

```
In [ ]: ## Reading metadata of the stations
stations_metadata = pd.read_csv('lab1/molene/weather_stations_filtered_Molene.csv', index_col=0)
stations_metadata
```

```
In [ ]: ## extracting the coordinates for each station from the metadata => these coordinates can be used to build a graph
coordinates_df = stations_metadata.iloc[:, -3:]
coordinates_df = coordinates_df - coordinates_df.mean() # => Removing mean to have more meaningful coordinates
coordinates = coordinates_df.to_numpy()
```

```
In [ ]: ## Add your code here
## You can PyGSP or NetwrokX package for this to have predefined graph builders.
```

Question 1

What type of graph did you use and what is the hyperparameter of it? Make a plot of the graph sparsity as a function of at least one hyper parameter. Make sure the graph remains connected.

Answer: Write your answer here ...

```
In [ ]: ## Add your code here
```

Noisy measurements

Let e be the average energy of the graph signals in the dataset.

$$e = \frac{1}{m} \sum_{i=1}^m \|\mathbf{x}_i\|_2^2,$$

where m is the number of data points. Generate m noisy graph signal $\mathbf{y} = \mathbf{x} + \mathbf{n}$ where $\mathbf{n}$ is a zero-mean Gaussian noise $\mathbf{n} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$. Consider $\sigma^2 = e/2$.

```
In [ ]: ## loading the data
timeseries_data = pd.read_csv('lab1/molene/filtered_data_Molene.csv', index_col=0)
timeseries_data = timeseries_data - timeseries_data.mean() # => removing mean from data
```

```
In [ ]: x = timeseries_data.to_numpy().transpose() # => transpose the data so we have each column as a signal
```

```
e = np.sum(np.square(x))/x.shape[1] # computing average energy

sigma = e/x.shape[0]/2
n = np.random.multivariate_normal(np.zeros(x.shape[0]),sigma*np.eye(x.shape[0]),size=x.s
y = x + n # adding to noise to synthesize noisy observations
```

Task 2

Use the Tikhonov regularizer for a choice of $\alpha > 0$ to recover signal $\mathbf{x}$ from the measurements $\mathbf{y}$. Provide the signal recovery metric root normalized mean square error (rNMSE)

$$\text{rNMSE} = \left(\frac{\sum_{i=1}^m \|\hat{\mathbf{x}}_i - \mathbf{x}_i\|_2^2}{\sum_{i=1}^m \|\mathbf{y}_i\|_2^2} \right)^{0.5}$$

```
In [ ]: ## set the regularizer coefficient
alpha = ...

## exploit the graph Laplacian (if you are using pygsp, convert it to dense format from
L = ...

## write down the closed form solution for Tikhonov regularizer
x_hat = np.matmul(np.linalg.inv(...),y)

rnmse = np.sqrt(np.square(x_hat-x).mean())/y.std() # rnmse definition

print(f"The rNMSE is: {np.round(rnmse,4)}")
```

Question 2

Illustrate rNMSE as a function of $\alpha \in [0, 10]$. What do you conclude about the role of the regularizer?

```
In [ ]: ## Add your code here
```

Answer: Write your answer here ...

Question 3

Consider now different noisy scenarios, $\sigma^2 = e/10$ and $\sigma^2 = e$. Add the respective line into your plot. What do you conclude about the role of the regularizer?

```
In [ ]: ## Add your code here
```

Answer: Write your answer here ...

Task 3

Implement trend filtering regularizer and compare the performance with the Tikhonov regularizer for $\sigma^2 = e/2$.

Write the regularizer of your choice here

```
In [ ]: ## Add your code here.  
## You may need an optimization solver if your chosen regularizer does not have a closed
```

Question 4

Why and when does one regularizer performs better than the other?

Answer: Write your answer here ...

Missing measurements

Consider now that the signal is noiseless but it is observed only in a portion of the nodes: $\mathbf{y} = \mathbf{P}\mathbf{x}$, where $\mathbf{P} \in \mathbb{R}^{M \times N}$ is the sampling matrix collecting M out of the possible N samples.

Task 4

Taking the best regularization weights from the above exercise, show the rNMSE of the recovered signal for both regularizers and $M = \{3, 9, 15, 21, 27\}$.

```
In [ ]: ## add your code here  
## Generate the data with missing values (or only indices).
```

```
In [ ]: ## Add your code here  
## solve and evaluate for Tikhonov regularizer
```

```
In [ ]: ## Add your code here  
## solve and evaluate for the regularizer of your choice
```

Question 5

What would you conclude about the behaviour of the two regularizers?

Answer: Write your answer here ...

Lab 1.2: Node Embedding, Node Classification, and Link Prediction

Objectives:

- Implement different traditional and embedding-based graph representation learning methods
- Compare and analyse the performances of the methods for embedding learning, node classification and link prediction tasks.

Completion requirements:

By the end of this notebook, you should have:

- Implemented all code cells for:
 - the representation learning methods.

- training the methods.
- assessing the performance of the methods.
- Answered the analysis questions on each section.

Libraries

To run this notebook you need to have installed the following packages:

- TDC
- Numpy
- Pytorch
- NetworkX
- Matplotlib
- Scikit-learn
- Pytorch geometric

Run the following code to import the packages

```
In [ ]: # import the libraries
import torch
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# import the functions
from numpy import dot
import torch.nn as nn
from sklearn import metrics
from numpy.linalg import norm
import torch.nn.functional as F
from sklearn.metrics import roc_auc_score
from torch_geometric.utils import mask as Mask
from torch_geometric.utils import to_networkx
from torch_geometric.nn.models import Node2Vec
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split

# import the datasets
from tdc.multi_pred import PPI
from torch_geometric.datasets import Planetoid
```

Lab 1.2.1: Generate and visualize the node embeddings

Decription of the dataset:

The Cora dataset consists of 2708 scientific publications classified into one of seven classes. The citation network consists of 5429 links. Each publication in the dataset is described by a 0/1-valued word vector indicating the absence/presence of the corresponding word from the dictionary. The dictionary consists of 1433 unique words. More infos about Cora dataset can be found here:

<https://paperswithcode.com/dataset/cora>.

Run the following code to load the cora dataset:

```
In [ ]: cora_dataset = Planetoid(root="", name="Cora", split="public")
data = cora_dataset[0]
```

Run the following code to check the characteristics of the dataset:

```
In [ ]: # number of nodes, dimensions of features, number of edges
print(data.x.shape[0])
print(data.x.shape[1])
print(data.edge_index.shape[1])
```

Question

(a) Generate hand crafted features by concatenating the

- 1. Degree centrality
- 2. clustering coefficient
- 3. betweenness centrality

as feature vector for each node and visualize it using TSNE.

```
In [ ]: # The package networkx has the corresponding function

#####
# write code here to convert the pytorch graph to networkx graph

# call the corresponding functions to calculate the hand crafted features

# Concatenate the features to be a feature vector

#####

#####
#visualize features

#####
```

Question

(b) Generate features vectors from Node2Vec using the following parameters. The output features should have three dimensions. Visualize it using TSNE.

The size of each embedding vector.

- embedding_dim=3

The walk length

- walk_length=20

The actual context size which is considered for positive samples.

This parameter increases the effective sampling rate by reusing samples across different source nodes.

- context_size=10

The number of walks to sample for each node

- walks_per_node=10

The number of negative samples to use for each positive sample

- num_negative_samples=1

Likelihood of immediately revisiting a node in the walk

- p=1

Control parameter to interpolate between breadth-first strategy and depth-first strategy

- q=1

If set to :obj:`True`, gradients w.r.t. to the weight matrix will be sparse

- sparse=False

batch size

- batch_size=128

shuffle the input data

- shuffle=True

number of epochs in training

- train_epochs=200

learning rate in training

- learning_rate=0.01

```
In [ ]: # The Node2Vec model is imported above from torch_geometric.nn.models

#####
# Define your Node2Vec model here

# Define a loader here

# define an optimizer here

#####

#####
#define the training function for the Node2Vec model here

#####

#####
#define the test function for the Node2Vec model here

#####
```

```
In [ ]: #####
# train your Node2Vec model

#####
```

```
In [ ]: #####
# produce embedding using the trained model

# visualize the features here

#####
```

Question

(c) Compare the features generated from above. From the visualization, which feature vector is more informative? And give the reasons in the following cell:

Answer here:

Lab 1.2.2: Node Classification

Problem definition

Node Classification is a machine learning task in graph-based data analysis, where the goal is to assign labels to nodes in a graph based on the properties of nodes and the relationships between them.

Node Classification models aim to predict non-existing node properties (known as the target property) based on other node properties. Model performance can be measured using benchmark datasets like Cora, Citeseer, and Pubmed, among others, typically using Accuracy and F1. In this exercise, we use accuracy.

The dataset needs to be split for the node classification task, and the performance of the model is measured by its accuracy on the test set. Cora has the built in split. We will use its default split in this exercise.

Question

(a) Use the hand-crafted features as input, perform node classification using a 2-layer Multi-Layer Perceptron (MLP).

The hyperparameters for the MLP should be as follows:

- the input dimensions are same as the dimensions of input features
- hidden unit dimension is set to be 16
- output dimensions should be the number of classes to predict
- use relu as activate function
- train the MLP for 200 epochs

Which loss would you use for training the MLP? Answer in the following cell:

Answer here:

In the following cell, define a two layer MLP using the parameters given above:

```
In [ ]: #####  
# define a two layer MLP  
  
#####
```

In the following cell, train your MLP for 200 epochs and print the training loss, accuracy on the test set

for each epoch:

```
In [ ]: #####
# instantiate your MLP model here

#####

#####
# define a training function

#####

#####
# define a test function

#####

#####
#train your model here and print the accuracy

#####
```

In the following cell, visualize the accuracy on the test set in each epoch:

```
In [ ]: #####
# visualize the accuracy on the test set as a function to the epochs

# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(test_acc_list, 'b-', label="Test Accuracy")

# Add a legend to the plot
ax.legend(loc='lower right')

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Test Accuracy')
ax.set_title('Test Accuracy')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
```



Question

(b) Repeat the node classification task using features generated from Node2Vec above

```
In [ ]: #####
# instantiate another MLP with the same parameters
# MLP node using node2vec emb

#####
```

```
#####
# get the features generated from the Node2Vec trained above
#####

#####

#####
# train function for MLP
#####

#####
# test function for your MLP
#####
```

```
In [ ]: #####
# train your model here:
```

```
#####
```

```
In [ ]: #####
# visualize the accuracy on the test set as a function to the epochs
```

```
# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(test_acc_list, 'b-', label='Test Accuracy')

# Add a legend to the plot
ax.legend(loc='lower right')

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Test Accuracy')
ax.set_title('Test Accuracy during training')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
#####
```

Question

(c) Repeat the node classification task using original features from Cora dataset

```
In [ ]: #####
# instantiate another MLP with the same parameters
# MLP node using cora features
```

```
#####
```

```
#####
# define a train function for your model
```

```
#####
```

```
#####
#define a test function to your model

#####
```

```
In [ ]: #####
# train your model

#####
```

```
In [ ]: #####
# visualize the accuracy on the test set as a function to the epochs

# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(test_acc_list, 'b-', label='Test Accuracy')

# Add a legend to the plot
ax.legend(loc='lower right')

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Test Accuracy')
ax.set_title('Test Accuracy during training')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
#####
```

Question

(d)Run label propagation algorithm on Cora for 10 iterations. Specifically use the row normalized adjacency matrix as the transition matrix. You can pad the label vectors for the test nodes using zero padding as initialization.

```
In [ ]: #####
# one-hot encode the true label for training data

#####

#####
# mask the val and test labels

#####

#####
# transition matrix as adj matrix

# row normalization

#####

#####
```

```
# write code here to run the label propagation algorithm for 10 epochs  
#####
```



Question

(e)(1) Which method performs the best? Give the answer in the following cell:

Answer:

(e)(2) What do you conclude about the informativeness of the extracted node feature? Give the answer in the following cell:

Answer:

(e)(3) Is there any particular method or embedding quality you would like to comment? Give the answer in the following cell:

Answer:



Question

(f) Increase the training set to 80% using one random division and run the above experiments again using hand-crafted features, Node2Vec generated features, original cora features, and label propagation algorithm. Run the code in the following cell to generate a random split.

```
In [ ]: num_nodes = data.x.shape[0]  
node_ids = [i for i in range(num_nodes)]  
  
# split the nodes into train-val-test: 80-10-10  
ids_train, ids_val_test = train_test_split(node_ids, test_size=0.2, random_state=42)  
ids_val, ids_test = train_test_split(ids_val_test, test_size=0.5, random_state=41)  
train_mask = Mask.index_to_mask(torch.Tensor(ids_train).long(), size=num_nodes)  
val_mask = Mask.index_to_mask(torch.Tensor(ids_val).long(), size=num_nodes)  
test_mask = Mask.index_to_mask(torch.Tensor(ids_test).long(), size=num_nodes)
```

```
In [ ]: #####  
# add your code here to run the experiment using hand-crafted features  
  
#####
```

```
In [ ]: #####  
# visualize the accuracy on the test set as a function to the epochs  
  
# Set the font size for the plot labels and title  
plt.rcParams.update({'font.size': 12})  
  
# Create a figure and axis object  
fig, ax = plt.subplots()  
  
# Plot the training loss as a blue line  
ax.plot(test_acc_list, 'b-', label='Test Accuracy')  
  
# Add a legend to the plot  
ax.legend(loc='lower right')
```

```

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Test Accuracy')
ax.set_title('Test Accuracy during training')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
#####

```

```

In [ ]: #####
# add your code here to run the experiment using Node2Vec features
# MLP node using node2vec emb

#####

```

```

In [ ]: #####
# visualize the accuracy on the test set as a function to the epochs

# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(test_acc_list, 'b-', label='Test Accuracy')

# Add a legend to the plot
ax.legend(loc='lower right')

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Test Accuracy')
ax.set_title('Test Accuracy during training')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
#####

```

```

In [ ]: #####
# add your code here to run the experiment using original cora features

#####

```

```

In [ ]: #####
# visualize the accuracy on the test set as a function to the epochs

# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(test_acc_list, 'b-', label='Test Accuracy')

# Add a legend to the plot
ax.legend(loc='lower right')

```

```
# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Test Accuracy')
ax.set_title('Test Accuracy during training')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
#####
```

```
In [ ]: #####
# add your code here to run the label propagation

#####
```



Question

(g) Compare the performance of different methods. Which method is affected most by the increase in training set size? Explain the possible reasons behind your observation in the following cell:

Answer:

Lab 1.2.3 Link Prediction



Description of the dataset:

Protein-protein interactions (PPI) are very important to discover new putative therapeutic targets to cure disease. Expensive and time-consuming wet-lab results are usually required to obtain PPI activity. PPI prediction aims to predict the PPI activity given a pair of proteins' amino acid sequences. For the Protein-protein interaction dataset, we build up the network as follows:

- Each protein is a node on an undirected graph G .
- There is a connection between the two nodes if there is a known interaction in the training data.
- The link prediction problem in this case is defined as: given the two nodes on the undirected graph, predict whether there is a link between them.

More information about the used PPI dataset can be found here:

https://tdcommons.ai/multi_pred_tasks/ppi/

Use 70% of the data as training data, 30% as test data. Run the following code to load and split the cora dataset:

```
In [ ]: ppi = 'HuRI'
ppi_data = PPI(name=ppi)
# negative sample using fraction=1
```

```

ppi_data.neg_sample(frac=1)
# This would return a pytorch-geometric graph
graph = ppi_data.to_graph(format='pyg', split=True, seed=42)
graph_pyg = graph['pyg_graph']
# common neighbors
num_nodes = graph_pyg.x.shape[0]
n_id = range(num_nodes)
# split the data using
split = ppi_data.get_split(method = 'random', seed = 42, frac = [0.7, 0.0, 0.3])

```

Run the following code to check the test set

```
In [ ]: split["test"]
```

As the positive and negative edges in the split are noted with the name of the proteins, run the following code to convert the positive and negative edges with the proteins name to the edges with their indices.

```

In [ ]: # convert the edges split into graph_pyg
map_book = graph["index_to_entities"]

train_edge_0 = split["train"]["Protein1_ID"].tolist()
train_edge_0_id = map(lambda key: map_book[key], train_edge_0)
train_edge_0 = [i for i in train_edge_0_id]

train_edge_1 = split["train"]["Protein2_ID"].tolist()
train_edge_1_id = map(lambda key: map_book[key], train_edge_1)
train_edge_1 = [i for i in train_edge_1_id]

train_edges = torch.vstack((torch.tensor(train_edge_0), torch.tensor(train_edge_1)))

test_edge_0 = split["test"]["Protein1_ID"].tolist()
test_edge_0_id = map(lambda key: map_book[key], test_edge_0)
test_edge_0 = [i for i in test_edge_0_id]

test_edge_1 = split["test"]["Protein2_ID"].tolist()
test_edge_1_id = map(lambda key: map_book[key], test_edge_1)
test_edge_1 = [i for i in test_edge_1_id]

test_edges = torch.vstack((torch.tensor(test_edge_0), torch.tensor(test_edge_1)))

```



Decription of the Metrics:

We use Average precision(AP) and Compute Area Under the Receiver Operating Characteristic Curve(AUROC) to evaluate the performance of the models in the link prediction task. The link to the descriptions of the metric can be found here: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html, https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html



Question

(a) For any pair of nodes (u,v) in the test set, compute scores for the link based on the common neighbors of the node pair normalized by the sigmoid function. Plot the ROC curve and the precision recall curve.

```
In [ ]: #####
```

```
#add your code here to calculate the common neighbors of the node pairs in the test set

#normalize the number of common neighbors using the sigmoid function

#####
```

```
In [ ]: #####
#add your code here to plot the ROC curve and precision recall curve

#####
```

```
In [ ]: #####
#add your code here to plot the precision recall curve

#####
```

Question

(b) Extract node features using Node2Vec. Parameters stay the same as in the previous experiment. Compute cosine similarity between the extracted embedding vectors of nodes u and v in the test set as the link score. Convert the similarities into a probability distribution by applying the sigmoid function. Plot the ROC curve and the precision recall curve.

```
In [ ]: #####
# add your code here to generate embeddings for the PPI dataset using Node2Vec

#####
```

```
In [ ]: #####
# add your code here to calculate the cosine simlairity of the node pairs in the test s
# dont forget to normalize the similarities using sigmoid

#####
```

```
In [ ]: #####
# add your code here to plot the roc curve

#####
```

```
In [ ]: #####
# add your code here to plot the precision and recall curve

#####
```

Question

(c) Which method performs better: Node2Vec or common neighbors? Give your answer and explain the reasons in the folowing cell.

Answer:

Question

(d) So far we have not used the negative edges of the train set during training. Instead of directly

computing scores from extracted features we will now train a two layer multilayer perceptron for predicting a link between two nodes. For any pair of nodes (u,v), concatenate the Node2Vec extracted embedding vectors of nodes u and v. The concatenated pair serve as input to the MLP. The output layer will be two neurons predicting the probability of existence of a link. Set the hidden layer dimension to 16.

In order to do the link prediction task, we need to prepare the data. Run the code in the following cell to concatenate the embeddings of the node pair and the true labels of the link.

```
In [ ]: # prepare data for link prediction task
train_emb = []
for e in torch.transpose(train_edges, 0, 1):
    emb1 = emb_n2v[e[0]]
    emb2 = emb_n2v[e[1]]
    edge_emb = torch.hstack((emb1, emb2))
    train_emb.append(edge_emb)
train_emb = torch.stack(train_emb)

test_emb = []
for e in torch.transpose(test_edges, 0, 1):
    emb1 = emb_n2v[e[0]]
    emb2 = emb_n2v[e[1]]
    edge_emb = torch.hstack((emb1, emb2))
    test_emb.append(edge_emb)
test_emb = torch.stack(test_emb)

train_y = torch.tensor(split["train"]["Y"])
test_y = torch.tensor(split["test"]["Y"])
```

In the following cell, instantiate a two layer MLP and train it using the training data prepared in the previous cell. Print and visualize the ap score and the auroc score as a function to the epochs

```
In [ ]: #####
#define and train your MLP here. print the scores during training

#####
```

```
In [ ]: #####
# visualize the scores as a function to the epochs

# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(aps, 'b-', label='Average Precision')
ax.plot(aurocs, 'r-', label='AUROC')

# Add a legend to the plot
ax.legend(loc='lower right')

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Metrics')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
```

```
plt.show()
#####
```

Question

(e) Now increase the fraction of negative edges in the train/val/test sets to 2. Run the MLP experiment again.

Run the following code to increase fraction of the negative edges and prepare the data again

```
In [ ]: ppi = 'HuRI'
ppi_data = PPI(name=ppi)
ppi_data.neg_sample(frac=2)
# This would return a pytorch-geometric graph
graph = ppi_data.to_graph(format='pyg', split=True, seed=42)
graph_pyg = graph['pyg_graph']
# common neighbors
num_nodes = graph_pyg.x.shape[0]
n_id = range(num_nodes)

split = ppi_data.get_split(method = 'random', seed = 42, frac = [0.7, 0.0, 0.3])

# convert the edges split into graph_pyg
map_book = graph["index_to_entities"]

train_edge_0 = split["train"]["Protein1_ID"].tolist()
train_edge_0_id = map(lambda key: map_book[key], train_edge_0)
train_edge_0 = [i for i in train_edge_0_id]

train_edge_1 = split["train"]["Protein2_ID"].tolist()
train_edge_1_id = map(lambda key: map_book[key], train_edge_1)
train_edge_1 = [i for i in train_edge_1_id]

train_edges = torch.vstack((torch.tensor(train_edge_0), torch.tensor(train_edge_1)))

test_edge_0 = split["test"]["Protein1_ID"].tolist()
test_edge_0_id = map(lambda key: map_book[key], test_edge_0)
test_edge_0 = [i for i in test_edge_0_id]

test_edge_1 = split["test"]["Protein2_ID"].tolist()
test_edge_1_id = map(lambda key: map_book[key], test_edge_1)
test_edge_1 = [i for i in test_edge_1_id]

test_edges = torch.vstack((torch.tensor(test_edge_0), torch.tensor(test_edge_1)))

# prepare data
model.eval()
emb_n2v = model(torch.arange(graph_pyg.num_nodes))
emb_n2v = emb_n2v.detach()
train_emb = []
for e in torch.transpose(train_edges, 0, 1):
    emb1 = emb_n2v[e[0]]
    emb2 = emb_n2v[e[1]]
    edge_emb = torch.hstack((emb1, emb2))
    train_emb.append(edge_emb)
train_emb = torch.stack(train_emb)

test_emb = []
for e in torch.transpose(test_edges, 0, 1):
    emb1 = emb_n2v[e[0]]
    emb2 = emb_n2v[e[1]]
```

```

edge_emb = torch.hstack((emb1, emb2))
test_emb.append(edge_emb)
test_emb = torch.stack(test_emb)

train_y = torch.tensor(split["train"]["Y"])
test_y = torch.tensor(split["test"]["Y"])

```

```

In [ ]: #####
# define and train your MLP here. print the scores during training

```

```

In [ ]: #####
# visualize the scores as a function to the epochs

# Set the font size for the plot labels and title
plt.rcParams.update({'font.size': 12})

# Create a figure and axis object
fig, ax = plt.subplots()

# Plot the training loss as a blue line
ax.plot(aps, 'b-', label='Average Precision')
ax.plot(aurocs, 'r-', label='AUROC')

# Add a legend to the plot
ax.legend(loc='lower right')

# Set the plot labels and title
ax.set_xlabel('Epoch')
ax.set_ylabel('Metrics')

# Add grid lines to the plot
ax.grid(True, which='both')

# Display the plot
plt.show()
#####

```



Question

(f) Which evaluation metrics is affected most by the imbalance in the size of the positive and the negative class? Which metrics would you then prefer to use in such an imbalanced setting and why?

Answer here:
